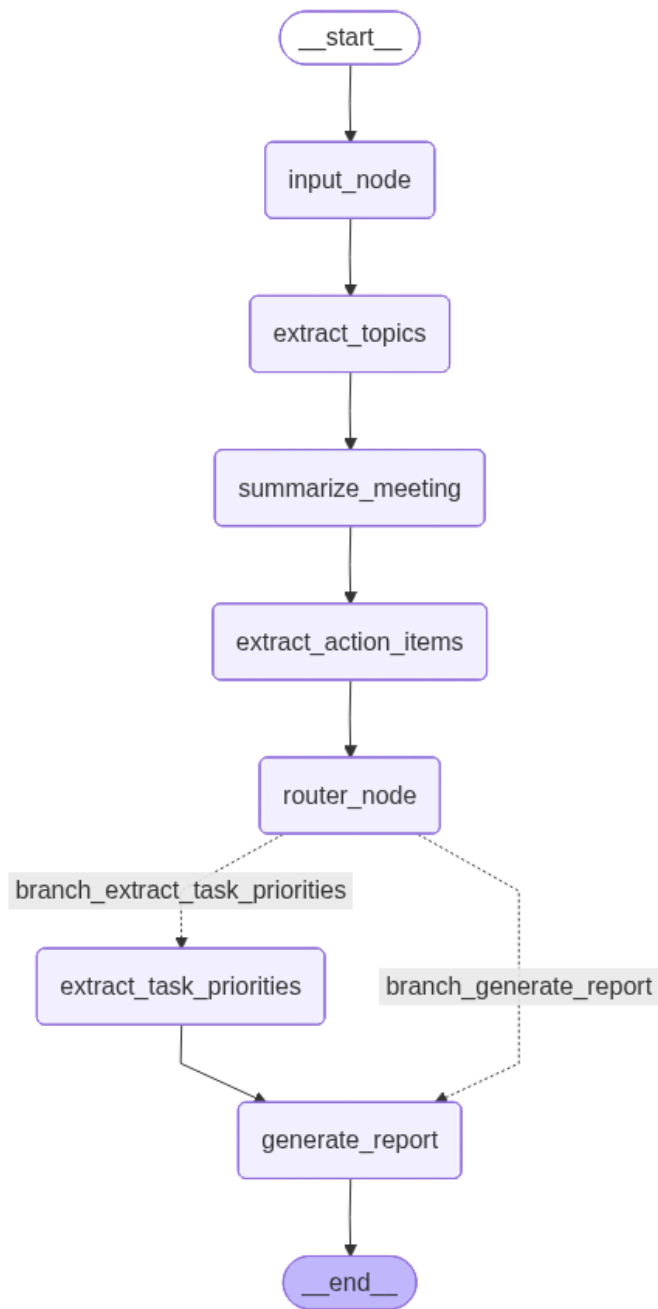


Project Proposal

1. **AI Meeting Notes Analyzer using langgraph.**
2. **Brief on the project:** This is the meeting notes analyzer of meeting happened between different stake holders. This project goes through transcript of a meeting, extracts key information from it, like key points, action items, priorities, deadlines etc. In this project there different AI agents created which extract a part of information they are responsible for, then sends the extracted information along with transcript, to the next node for processing.
3. **Deliverables of the project:** As part to approach this project, we create different nodes using langgraph. These nodes act as individual agents, responsible for extracting specific part of information.
 - We have created input_node, extract_topics node, summarize_meeting node, extract_action_item node, router node for branching, extract_task_priorities_node and generate_report node.
 - Details of the model , important findings, expecting observations and outcome. We have used two different LLM models to generate responses from agents, gpt-5-mini from OpenAI and gemini-2.5-flash-light.
 - Given below is the screenshot of agentic ai workflow.



4. Resources

- **Data set source:** We got the transcript as input within the project itself.
- **Software:** Software you will choose to solve the problem.
 - a) Python
 - b) Python-dotenv
 - c) Langchain-core
 - d) Langchain-openai
 - e) Langchain-google-genai
 - f) Langchain-experimental

- **References:** Include 1-3 relevant papers which already discussed same/similar problems. (not compulsory)

5. **Individual Details:**

Name: Wasim Bari

Mobile: 9650307130

E-mail: wbari31@gmail.com

6. **Milestones**

Milestones
Define a problem
Understanding the business problem
Get the Data
Explore and pre-process data
Choosing the python platform
Create Features
EDA
Create Model
Model Evaluation
Report Writing
Project submission

Defining a problem:

There is a meeting transcript given for meeting happened about issue encountered in android app. As a meeting note analyzer, this app is supposed to summarize the meeting, key insights, action items, owners, priorities so that someone who missed the meeting can get the details about the meeting in real quick.

Understanding the business problem:

This is helpful for someone who needs to get brief idea about the meeting, without having to play the meeting in full. This saves on time of the user, also highlights what important happened during the meeting along with key take aways. This can further be integrated with meeting apps like zoom or teams to generate the meeting summary after every meeting and send it to all participants.

Getting data:

Data is provided with the problem statement.

Agentic AI workflow code:

- Loading Open AI gpt model:

```
# OpenAI gpt model as llm
llm=ChatOpenAI(model="gpt-5-mini", temperature=1.0)
```

- Creating python class to store state.

```
# Python class to store states
class GraphState (TypedDict):
    transcript: str
    topics_list: list[str]
    meeting_summary: str
    task_assignees: list[tuple[str, str]]
    task_priorities: list[tuple[str, str]]
    final_report: str
```

- Creating input node:

```
# input node to read input transcript
def input_node(state: GraphState) -> GraphState:
    state['transcript'] = input_transcript
    return state
```

- Creating topic extraction node:

```
# Topic extraction node
def extract_topics(state: GraphState) -> GraphState:
    transcript = state['transcript']
    topics = llm.invoke(f"""Extract the main topics from the meeting
transcript. Max 3 topics.
Topics are separated by new lines.

Input is transcript.

Return value is string.

Transcript:\n\n{transcript}""")
    #print(f'extract_topics. topics:{topics}')
    #print(type(topics))
    state['topics_list'] = topics.content.split('\n') # Assuming
topics are separated by newlines
    return state
```

- Creating meeting summarization node:

```

# Meeting summarization node
def summarize_meeting(state: GraphState) -> GraphState:
    transcript = state['transcript']
    summary = llm.invoke(f"""Summarize the following meeting transcript in
a concise manner. Summarize in
    3-4 sentences.

    Input is transcript.

    Return value is string.

    Transcript:\n\n{transcript}""")
    state['meeting_summary'] = summary.content
    return state

```

- **Creating action item extraction node:**

```

# Action items extraction node
def extract_action_items(state: GraphState) -> GraphState:
    transcript = state['transcript']
    action_items = llm.invoke(f"""Extract the action items from the
meeting transcript. An action item consists
    of a task and an assignee. Format the output as a list of tuples with
first element 'task' and
    second element 'assignee'. If no assignee found for a task, assign
'Not Specified'. If no action items found,
    return empty list.

    Input is transcript.

    Return value is a list of tuples.

    Transcript:\n\n{transcript}""")

    state['task_assignees'] = action_items.content
    return state

```

- **Task priority extraction node:**

```

# Task Priority extraction node
def extract_task_priorities(state: GraphState) -> GraphState:
    transcript = state['transcript']
    task_priorities = llm.invoke(f"""Extract the task priorities from the
meeting transcript. A task priority
    consists of a task and its priority level (High, Medium, Low). Format
the output as a list of tuples

```

```

        with first element 'task' and second element 'priority'. If no
priority found for a task, assign
        'Not Specified'. Inputs are transcript and task_assignees which is a
list of tuples.
        Return value is a list of tuples.
        Transcript:\n\n{transcript}""")

    state['task_priorities'] = task_priorities.content
    return state

```

- Report generation node:

```

# Report generation node
def generate_report(state: GraphState) -> GraphState:
    meeting_summary = state['meeting_summary']
    task_assignees = state['task_assignees']
    task_priorities = state['task_priorities']
    topics_list = state['topics_list']

    report = llm.invoke(f"""Generate a comprehensive report based on the
meeting summary, action items, topics
list and task priorities.
The report should include the meeting summary followed by topics list
and then list of action items
with their respective assignees and priority levels.

Input is meeting_summary, task_assignees, topics_list and
task_priorities. Return value is a string.
Meeting Summary:\n\n{meeting_summary}\n\nAction
Items:\n\n{task_assignees}\n\nTask Priorities:\n\n{task_priorities}""")

    state['final_report'] = report.content
    return state

```

- Router node:

```

# router node for conditional routing.
def router_node(state: GraphState) -> GraphState:
    #print(f'router_node. state:{state}')
    #return {'task_assignees': state['task_assignees']}
    return state

```

- Router function:

```

# routing function. This function will be used by the router node to
decide which route to take.

```

```
def routing_function(state: GraphState) -> str:
    task_assignees = state['task_assignees']
    if task_assignees:
        return 'branch_extract_task_priorities'
    return 'branch_generate_report'
```

- Stategraph workflows:

```
# Defining workflows
workflow=StateGraph(GraphState)
workflow.add_node('input_node',input_node)
workflow.add_node('extract_topics',extract_topics)
workflow.add_node('summarize_meeting',summarize_meeting)
workflow.add_node('extract_action_items',extract_action_items)
workflow.add_node('router_node',router_node)
workflow.add_node('extract_task_priorities',extract_task_priorities)
workflow.add_node('generate_report',generate_report)

workflow.add_edge(START, 'input_node')
workflow.add_edge('input_node', 'extract_topics')
workflow.add_edge('extract_topics', 'summarize_meeting')
workflow.add_edge('summarize_meeting', 'extract_action_items')
workflow.add_edge('extract_action_items', 'router_node')
workflow.add_conditional_edges(
    "router_node",
    routing_function,
    {
        "branch_extract_task_priorities": "extract_task_priorities",
        "branch_generate_report": "generate_report"
    }
)

workflow.add_edge('extract_task_priorities', 'generate_report')
workflow.add_edge('generate_report', END)

app=workflow.compile()
```

- Running state graph workflow using gpt llm.

```
# Invoking workflow graph using gpt model and displaying final report.
state=app.invoke({'transcript': input_transcript})
print(state['final_report'])
```

- Changing llm model to use gemini 2.5 flash lite model:

```
# Setting gemini as llm.
```

```
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash-lite",
temperature=1.0)
```

- Running state graph workflow using gemini model:

```
# Invoking workflow graph using gemini model and displaying final report.
state=app.invoke({'transcript': input_transcript})
print(state['final_report'])
```

- Stategraph workflow execution result using gpt model:

Meeting Summary:

The team addressed an urgent issue where the mobile app has been crashing during login for many users (120+ complaints) since an authentication service update three days ago, with crashes occurring mainly on Android. Developers suspect a changed API response or missing error handling in the Android login module. Backend will review logs and API; mobile dev will implement fixes and improve error handling; QA will reproduce and test on Android 11-13 and prepare a bug report. Customer Support will inform users and management. The team aims to release a patched Play Store update after regression testing and will regroup tomorrow to review progress.

Topics List:

- Issue overview and impact (120+ crash reports; Android-focused)
- Suspected causes (changed API response; missing error handling in Android login module)
- Immediate developer actions (mobile fixes, backend log/API review)
- QA plan (reproduce on Android 11-13, bug report, regression testing)
- Backend verification (check API response format, logs, test endpoints)
- Communication (customer support notifications, management update)
- Release plan (patch release to Play Store after regression testing)
- Timeline and checkpoints (target fix by tomorrow if in login module; regroup tomorrow morning)
- Documentation and tracking (bug report, issue tracker updates)

Action Items (with Assignees and Priority):

- Review the Android login module and check for missing error handling
 - Assignee: Mobile Developer
 - Priority: High
- Start working on the fix immediately
 - Assignee: Mobile Developer
 - Priority: High
- Push a fix by tomorrow (if issue is in the login module)
 - Assignee: Mobile Developer
 - Priority: High
 - Note: Target deadline indicated in meeting.

- Add better error handling so the app does not crash if the API fails
 - Assignee: Mobile Developer
 - Priority: High
- Provide an update by the end of the day
 - Assignee: Mobile Developer
 - Priority: High
- Prepare a detailed bug report with screenshots and logs
 - Assignee: QA Tester
 - Priority: High
- Share the bug report within the next hour
 - Assignee: QA Tester
 - Priority: High
 - Note: Time-sensitive to enable immediate dev/backend work.
- Test the login flow on Android 11, 12, and 13
 - Assignee: QA Tester
 - Priority: High
- Perform regression testing once the fix is ready
 - Assignee: QA Tester
 - Priority: High
- Update the issue tracker with all findings
 - Assignee: QA Tester
 - Priority: Not Specified
- Release the patch update to the Play Store after testing
 - Assignee: Not Specified
 - Priority: Not Specified
- Verify the authentication API and check for breaking changes
 - Assignee: Backend Developer
 - Priority: High
- Review server/API logs for authentication errors
 - Assignee: Backend Developer
 - Priority: High
- Test authentication endpoints after the fix is deployed
 - Assignee: Backend Developer
 - Priority: High
- Provide an update by the end of the day and report any issues found
 - Assignee: Backend Developer
 - Priority: High

- Send a message to the support team so they can update customers that we are working on a fix
 - Assignee: Customer Support Lead
 - Priority: Not Specified
- Prioritize this issue today
 - Assignee: Not Specified
 - Priority: Not Specified
- Inform the management team about this issue
 - Assignee: Not Specified
 - Priority: Not Specified
- Document the issue and the resolution steps
 - Assignee: Not Specified
 - Priority: Not Specified
- Regroup tomorrow morning to review progress
 - Assignee: Not Specified
 - Priority: Not Specified

Notes / Recommendations:

- Items marked High are time-sensitive and should be completed first (mobile fixes, backend verification, QA reproduction/reporting).
- Ensure the QA bug report and logs are shared immediately (within the hour) to unblock development and backend analysis.
- Assign owners for the "Not Specified" items (release to Play Store, management notification, prioritization, final documentation) if not already decided to avoid gaps in coordination.
- Confirm final patch release steps and responsible person for publishing to Play Store once regression testing passes.

- Stategraph workflow execution result using gemini model:

****Meeting Report: Mobile App Login Crashing Issue****

****Meeting Summary:****

The team convened to address a critical issue causing the mobile application to crash during the login process, predominantly affecting Android users. This instability is believed to be a direct consequence of a recent backend update to the authentication service, deployed approximately three days prior. Current efforts are focused on a dual investigation of both the mobile login module and the authentication API to pinpoint the root cause. The primary objective is to implement an urgent fix to mitigate customer frustration and prevent further impact. A patch update will be released following comprehensive testing, and

relevant stakeholders, including management and users, will be kept informed of the resolution.

****Topics Discussed:****

- * ****Mobile App Crashing During Login:**** The central issue being addressed, impacting Android users.
- * ****Backend Authentication Service Update:**** The suspected trigger for the crashing issue, deployed three days ago.
- * ****Investigation of Mobile Login Module:**** One of the key areas of focus for identifying the root cause.
- * ****Investigation of Authentication API:**** The other critical area being examined for the source of the problem.
- * ****Urgent Resolution of the Issue:**** The team's commitment to fixing the problem swiftly.
- * ****Patch Update Release:**** The planned deployment of a corrected version of the app.
- * ****Communication Strategy:**** Informing management and users about the issue and its resolution.

****Action Items:****

Task			
Assignee		Priority	
:-----			
----- :----- :-----			
Review the Android login module and check if any error handling is missing.			
Mobile Developer		High	
Prepare a detailed bug report with screenshots and logs.			
QA Tester		High	
Verify the authentication API and check if there are any breaking changes.			
Backend Developer		High	
Test the login flow on Android 11, 12, and 13.			
QA Tester		Medium	
Check the server logs to see if any authentication errors are recorded.			
Backend Developer		Medium	
Send a message to the support team so they can update customers that we are working on a fix.			
Customer Support Lead		Low	
Perform regression testing once the fix is ready.			
QA Tester		High	
Test the authentication endpoints after the fix is deployed.			
Backend Developer		High	
Document the issue and the resolution steps.			
Not Specified		Low	

Update the issue tracker with all findings.		
QA Tester	Low	
Notify users once the fix is deployed.		
Not Specified	Low	
Add better error handling so the app does not crash even if the API fails.	Mobile Developer	Medium
Release the patch update to the Play Store after confirming the issue is fully resolved.	QA Tester	High
Start working on the fix immediately after this meeting.		
Mobile Developer	High	
Review the API logs right away.		
Backend Developer	High	
Share the bug report within the next hour.		
QA Tester	High	
Provide an update by the end of the day.		
Mobile Developer	High	
Report any issues found.		
Backend Developer	High	